

01-Java基础

Java 基础面试题

1. 概念篇

问：说一下 Java 的特点？

答： – 平台无关性：源码编译为字节码，由 JVM 在各平台翻译执行，实现「一次编写，到处运行」 – 面向对象：封装、继承、多态，代码易维护和复用 – 自动内存管理：GC 自动回收无用对象，减少内存泄漏

问：Java 为什么是跨平台的？

答：Java 源码经 `javac` 编译为 `.class` 字节码，再由各平台的 JVM 翻译成机器码执行。

- 跨平台的是 **Java 程序（字节码）**，不是 JVM 本身
- JVM 用 C/C++ 编写，是平台相关的，不同系统需安装对应版本 JVM
- 字节码在各平台相同，JVM 负责适配底层差异

问：JVM、JDK、JRE 三者关系？

答：

组件	说明
JVM	Java 虚拟机，执行字节码，提供内存管理、GC
JRE	运行环境 = JVM + 核心类库，不含开发工具
JDK	开发工具包 = JRE + <code>javac</code> 、 <code>jdb</code> 等开发工具

关系：JDK ⊃ JRE ⊃ JVM

问：为什么 Java 既有编译又有解释？

答：1. 编译阶段：`javac` 将源码编译为字节码（平台无关） 2. 运行阶段：JVM 解释器逐条执行字节码；同时 JIT 将热点代码编译为本地机器码缓存

因此 Java 是 **编译 + 解释混合** 模式，兼顾跨平台与执行效率。

JIT 热点识别：JVM 通过方法调用计数器和回边计数器识别热点代码，达到阈值后编译为本地机器码缓存到 Code Cache，后续直接执行机器码。

问：JVM 和 Java 有什么区别？

答： – **Java**：编程语言，提供语法、关键字、标准类库（String、List 等） – **JVM**：运行平台，负责加载字节码、翻译为机器码、内存管理与 GC

关系：JDK ⊃ JRE ⊃ JVM。跨平台能力由 JVM 提供，字节码可在 Windows/Linux/Mac 的 JVM 上运行。

补充：JVM 不只跑 Java，Kotlin、Scala 等编译后也是字节码，可在 JVM 上运行。

问：Java 的优势和劣势？

答： **优势**： – 跨平台（JVM） – 面向对象，生态成熟（Spring、MyBatis 等） – 自动 GC，内置多线程支持 – 企业级稳定性，向后兼容好 – 安全模型（沙箱机制）

劣势： – 性能不如 C++/Rust（JVM 有额外开销，启动慢） – 语法相对繁琐（样板代码多） – JVM 本身占内存 – 开发效率不如 Python 等动态语言

问：编译型语言和解释型语言的区别？

答：

类型	特点	代表
编译型	先整体编译为机器码/字节码再执行，速度快	C、C++
解释型	运行时逐行解释执行，跨平台好、速度较慢	Python、JavaScript
混合型	先编译为字节码，再解释 + JIT	Java

Java 严格说是 **编译 + 解释混合**：`javac` 编译为 `.class`，JVM 解释执行 + JIT 编译热点代码。

问：Java 是值传递还是引用传递？

答：Java 只有值传递。

- **基本类型**：传递值的副本，方法内修改不影响原变量
- **引用类型**：传递引用的副本（地址副本），可通过副本修改对象内容，但无法改变原引用的指向

2. 数据类型篇

问：八种基本数据类型是什么？

答：

类型	字节	默认值	包装类
byte	1	0	Byte
short	2	0	Short
int	4	0	Integer
long	8	0L	Long
float	4	0.0f	Float
double	8	0.0d	Double
char	2	'000'	Character
boolean	-	false	Boolean

问：int 和 Integer 的区别？Integer 缓存机制？

答：- `int` 是基本类型，存栈上（局部变量），默认值 0 - `Integer` 是包装类，是对象，可为 null

装箱/拆箱：自动装箱调用 `Integer.valueOf()`，拆箱调用 `intValue()`

缓存机制：`Integer.valueOf()` 对 `-128 ~ 127` 之间的值会复用缓存对象。在此范围内 `==` 可能为 true，超出范围比较的是对象地址。

```
Integer a = 127, b = 127; // a == b → true
Integer c = 128, d = 128; // c == d → false
```

为什么有包装类：泛型只能使用引用类型；集合框架需要对象；提供工具方法（如 `Integer.parseInt()`）。

为什么保留 `int`：性能更好（栈上分配、无装箱开销）；语义清晰（不可为 null 的数值计算）。

问：装箱和拆箱是什么？有什么坑？

答：- 装箱（Autoboxing）：基本类型 → 包装类，如 `Integer i = 10`，实际调用 `Integer.valueOf(10)`

- 拆箱（Unboxing）：包装类 → 基本类型，如 `int n = i`，实际调用 `i.intValue()`

常见坑：1. NPE：`Integer i = null; int n = i;` → 拆箱时 NPE 2. 性能：循环中频繁装箱/拆箱（如 `List<Integer>` 累加）产生大量临时对象 3. `==` 比较：超出缓存范围时 `==` 比较的是地址，应用 `equals` 4.

三目运算符：`true ? new Integer(1) : 2.0` 会拆箱为 double，可能 NPE

问：数据类型转换方式有哪些？会有什么问题？

答：

转换方式	说明	示例
自动转换（隐式）	小范围 → 大范围，安全	<code>int</code> → <code>long</code>
强制转换（显式）	大范围 → 小范围，可能丢失	<code>(int) 3.14</code> → 3
字符串转换	<code>Integer.parseInt()</code> 、 <code>Double.parseDouble()</code>	<code>"123"</code> → 123
向上转型	子类 → 父类，自动、安全	<code>Dog</code> → <code>Animal</code>
向下转型	父类 → 子类，需 <code>instanceof</code> 检查	否则 <code>ClassCastException</code>

风险： - 数据溢出：`byte b = (byte) 300;` → 44（截断高位） - 精度损失：`(int) 3.14` → 3 - `ClassCastException`: `Animal a = new Animal(); Dog d = (Dog) a;`

问：为什么用 `BigDecimal` 不用 `double`？

答：`float / double` 采用 IEEE 754 浮点标准，二进制无法精确表示部分十进制小数（如 0.1），存在精度丢失。金融计算应使用 `BigDecimal`，通过 `String` 构造避免精度问题。

```
// 错误: double 构造仍有精度问题
BigDecimal a = new BigDecimal(0.1); // 实际 0.100000000000000005551115...
// 正确
BigDecimal b = new BigDecimal("0.1");
BigDecimal sum = b.add(new BigDecimal("0.2")); // 0.3
```

运算注意：使用 `add`、`subtract`、`multiply`、`divide` 方法；除法需指定精度和舍入模式 `RoundingMode.HALF_UP`。

3. 面向对象篇

问：封装、继承、多态怎么理解？

答： - 封装：隐藏内部实现，通过 `public` 方法暴露接口 - 继承：子类复用父类属性和方法，`extends` 单继承 - 多态：同一引用调用同一方法，不同子类表现不同行为 - 编译看左边（引用类型），运行看右边（实际对象） - 体现：方法重写、接口实现、方法重载

问：多态体现在哪几个方面？

答： 1. 方法重载（编译期多态）：同类中同名不同参，编译器根据参数决定调用哪个 2. 方法重写（运行期多态）：子类覆盖父类方法，JVM 根据实际对象类型调用 3. 接口多态：多个类实现同一接口，用接口引用调用，表现不同实现 4. 向上/向下转型：父类引用指向子类对象；向下转型需 `instanceof` 检查

问：多态解决了什么问题？

答： – 扩展性：新增子类无需修改调用方代码（开闭原则） – 复用性：父类/接口定义统一 API，子类提供不同实现 – 解耦：面向接口编程，降低模块依赖

典型应用：策略模式、模板方法、依赖倒置、消除冗长 if-else、Spring 中接口注入不同实现。

问：面向对象的设计原则有哪些？

答：

原则	含义
单一职责（SRP）	一个类只负责一项职责
开闭原则（OCP）	对扩展开放，对修改封闭
里氏替换（LSP）	子类可替换父类，不改变程序行为
接口隔离（ISP）	接口小而专，不强迫依赖不需要的方法
依赖倒置（DIP）	高层不依赖低层，都依赖抽象
迪米特法则	只与直接朋友交互，减少耦合

问：重载和重写的区别？

答：

对比	重载（Overload）	重写（Override）
位置	同类中	子类与父类/接口
方法名	相同	相同
参数列表	必须不同	必须相同
返回类型	可不同	相同或子类（协变）
访问权限	无限制	不能更严格
绑定	编译期	运行期

问：抽象类和接口的区别？

答：

对比	抽象类	接口
关键字	abstract class	interface
构造方法	可以有	不能有
成员变量	任意修饰符	public static final
方法	抽象 + 普通方法	抽象（JDK8前） / default / static
继承	单继承	多实现
设计思想	is-a 关系	has-a / can-do 能力

问：抽象类和普通类的区别？

答：

对比	普通类	抽象类
实例化	可直接 new	不能直接 new
方法	可有具体实现	可有抽象方法 + 具体方法
构造器	有	有（供子类调用）
用途	完整类	作为基类被继承

问：抽象类能被实例化吗？能加 final 吗？

答： – 不能实例化：不能用 `new AbstractClass()`，但子类实例化时会调用抽象类构造器 – 不能加 final：抽象类必须被继承，final 禁止继承，两者互斥

问：接口里可以定义哪些方法？

答：

类型	JDK 版本	说明
抽象方法	1.0+	默认 public abstract，实现类必须实现
default 方法	8+	带方法体，实现类可重写
static 方法	8+	属于接口本身，接口名直接调用
private 方法	9+	辅助 default 方法，实现类不可见
常量	1.0+	public static final，必须赋初值

接口不能有构造方法。

4. String 与 Object 篇

问：String 为什么不可变？

答：1. final 修饰类，内部 char[] / byte[] 也被 final 修饰 2. 字符串常量池复用，不可变保证安全 3. 作为 HashMap 的 key，hash 值不变 4. 天然线程安全
任何「修改」都会创建新 String 对象。

问：String 常量池是什么？intern() 的作用？

答：字符串常量池（String Pool）位于堆中（JDK 7+ 从方法区移入堆），存储字面量和 intern 后的字符串，实现复用。

创建方式	说明
<code>String s = "abc"</code>	常量池查找/创建，直接引用
<code>new String("abc")</code>	堆中新建对象；若常量池无“abc”则同时创建常量池对象
<code>s.intern()</code>	将堆中字符串引用放入常量池，返回常量池引用

经典题：new String("abc") 创建几个对象？ - 常量池无“abc”：2 个（堆对象 + 常量池对象） - 常量池已有“abc”：1 个（仅堆对象）

```
String a = "hello", b = "hello"; // a == b → true (同一常量池对象)
String c = new String("hello"), d = new String("hello"); // c == d → false
```

JDK 9+ 优化：String 底层由 char[] 改为 byte[] + 编码标志（Compact Strings），Latin-1 字符占 1 字节。

问：String、StringBuffer、StringBuilder 区别？

答：

类	可变性	线程安全	性能	场景
String	不可变	安全	–	少量字符串
StringBuffer	可变	安全（synchronized）	较低	多线程拼接
StringBuilder	可变	不安全	最高	单线程拼接

问：equals 和 hashCode 的关系？为什么要一起重写？

答：规则：equals 相等 → hashCode 必须相等；hashCode 相等 → equals 不一定相等。

原因：HashMap/HashSet 先用 hashCode 定位桶，再用 equals 比较。只重写 equals 会导致相同对象落入不同桶，破坏集合语义。

问：== 和 equals 的区别？

答：– ==：基本类型比较值；引用类型比较内存地址 – equals：Object 默认等同 ==；String 等类重写后比较内容

问：Object 类有哪些核心方法？

答：

方法	作用
equals(Object)	默认比较地址，常需重写
hashCode()	返回哈希码，重写 equals 必须重写
toString()	默认 类名@哈希码，常重写便于调试
getClass()	返回运行时实际 Class，不可重写
clone()	浅拷贝，需实现 Cloneable
wait() / notify() / notifyAll()	线程协作，需在 synchronized 中使用
finalize()	GC 前回调，已废弃（JDK 9+），不推荐

问：Java 创建对象有哪些方式？

答：

方式	是否调用构造器	特点
new	是	最常用，直接明确
反射 Constructor.newInstance()	是	运行时动态创建，Spring IOC 核心
clone()	否	浅拷贝，需 Cloneable
反序列化	否	不调用构造器，需 Serializable
工厂方法	是（方法内）	Integer.valueOf()、单例工厂

注意：Class.newInstance() 已过时，推荐 getDeclaredConstructor().newInstance()。

5. 反射、注解、异常篇

问：什么是反射？应用场景？

答：运行时动态获取类的属性、方法、构造器等信息，并能动态创建对象、调用方法。

应用场景： – Spring IOC 容器创建 Bean – 动态代理（AOP） – JDBC 加载驱动 `Class.forName()` – 注解解析（`@Autowired`、`@RequestMapping`）

问：Java 注解的原理是什么？

答： – 注解本质是继承 `Annotation` 接口的特殊接口 – 编译后转为 `.class` 中的属性表（`RuntimeVisibleAnnotations` 等） – 运行时通过反射获取，返回的是 **动态代理对象**（`AnnotationInvocationHandler`） – 调用注解方法时，从 `memberValues` Map 中取值（来源于常量池）

元注解：

元注解	作用
<code>@Target</code>	指定作用范围（类、方法、字段等）
<code>@Retention</code>	保留策略：SOURCE / CLASS / RUNTIME
<code>@Documented</code>	是否出现在 javadoc
<code>@Inherited</code>	是否被继承
<code>@Repeatable</code>	是否可重复（Java 8+）

保留策略： – SOURCE：仅源码，编译后丢弃（如 `@Override`） – CLASS：保留到 `.class`，运行时不可见 – RUNTIME：运行时可通过反射读取（Spring、MyBatis 注解）

问：Java 异常体系？

答：

```
Throwable
├── Error (不可恢复, 如 OutOfMemoryError、StackOverflowError)
└── Exception
    ├── Checked Exception (编译期检查, 必须处理, 如 IOException)
    └── Unchecked Exception (RuntimeException, 如 NPE、IllegalArgumentException)
```

问：throw 和 throws 的区别？什么时候不用 throws？

答：

对比	throw	throws
位置	方法体内	方法签名
作用	主动抛出一个异常对象	声明方法可能抛出的异常，交给调用者处理
数量	一次抛一个	可声明多个

不用 throws 的情况：1. 抛出的是 RuntimeException 或其子类（非受检异常）2. 在方法内部 try-catch 捕获并处理了异常

问：try-catch-finally 中 return 执行顺序？

答：finally 一定执行（除非 System.exit()）。若 finally 中有 return，会覆盖 try/catch 中的返回值。

执行细节：1. try 中若 return，先计算返回值并暂存，再执行 finally 2. finally 中若有 return，覆盖 try/catch 的返回值 3. finally 中修改 try 中已确定的返回值变量，不影响最终返回值（基本类型/不可变对象）

```
try { return "a"; } finally { return "b"; } // 返回 "b"
```

6. Java 8 新特性篇

问：Java 8 有哪些新特性？

答：1. Lambda 表达式：(params) -> expression 2. Stream API：函数式数据处理，filter、map、reduce 3. Optional：避免 NPE 4. 接口默认方法：default 关键字 5. 新日期 API：LocalDateTime、ZonedDateTime 6. 方法引用：类名::方法名

问：Lambda 表达式是什么？使用条件？

答：Lambda 是匿名函数的简写，替代匿名内部类。

```
// 匿名内部类
Runnable r1 = new Runnable() { public void run() { System.out.println("hi"); } };
// Lambda
Runnable r2 = () -> System.out.println("hi");
```

使用条件：目标类型必须是 函数式接口（仅一个抽象方法），可用 @FunctionalInterface 标记。

常见函数式接口：Runnable、Comparator、Consumer<T>、Supplier<T>、Function<T,R>、Predicate<T>。

问：Stream API 常用操作有哪些？

答：Stream 提供声明式数据处理，支持链式调用和并行流。

类型	常用 API	说明
中间操作	<code>filter</code> 、 <code>map</code> 、 <code>flatMap</code> 、 <code>distinct</code> 、 <code>sorted</code> 、 <code>limit</code>	返回 Stream，可链式
终端操作	<code>collect</code> 、 <code>forEach</code> 、 <code>reduce</code> 、 <code>count</code> 、 <code>anyMatch</code>	触发计算，返回结果

```
List<String> result = list.stream()
    .filter(s -> s.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

注意：Stream 只能使用一次；中间操作是惰性的，终端操作才触发执行。

问：Optional 怎么用？能解决什么问题？

答：Optional 封装可能为 null 的值，避免 NPE，表达「值可能缺失」的语义。

```
Optional<String> opt = Optional.ofNullable(getName());
String name = opt.orElse("default");
opt.ifPresent(System.out::println);
String upper = opt.map(String::toUpperCase).orElse("UNKNOWN");
```

不要用 Optional 作为：方法参数、成员变量、集合元素（增加开销和复杂度）。

问：CompletableFuture 是什么？常用 API？

答：JDK 8 异步编程工具，支持非阻塞回调和多任务组合。

对比	Future	CompletableFuture
获取结果	<code>get()</code> 阻塞	回调 <code>thenApply</code> 非阻塞
链式组合	不支持	<code>thenCompose</code> 、 <code>thenApply</code>
多任务	不支持	<code>allOf</code> 、 <code>anyOf</code>
异常	需 try-catch	<code>exceptionally</code> 、 <code>handle</code>

```
CompletableFuture.supplyAsync(() -> queryDB(), executor)
    .thenApply(data -> transform(data))
    .exceptionally(ex -> defaultValue);
```

注意：生产环境应指定自定义线程池，避免默认 `ForkJoinPool.commonPool()` 被占满。

问：Stream 并行流注意什么？

答：– 适合计算密集型、数据量大的场景 – 共享数据结构需线程安全 – 避免有状态操作 – 默认使用 `ForkJoinPool.commonPool()`

7. 深拷贝与浅拷贝

问：深拷贝和浅拷贝的区别？

答：– 浅拷贝：只复制对象本身和基本类型字段，引用类型字段仍指向同一对象 – 深拷贝：递归复制所有引用对象，完全独立

实现方式：

方式	说明
<code>Cloneable + clone()</code>	默认浅拷贝，深拷贝需递归 clone 引用字段
序列化/反序列化	通过 <code>ObjectOutputStream/ObjectInputStream</code> ，需 <code>Serializable</code>
手动递归复制	逐字段复制，最可控
第三方库	<code>MapStruct</code> 、深拷贝工具； <code>Apache BeanUtils</code> 多为浅拷贝

8. 泛型

问：Java 泛型的原理是什么？什么是类型擦除？

答：Java 泛型是 编译期语法糖，编译后泛型信息被擦除，字节码中只保留原始类型（Raw Type）。

- `List<String>` 编译后变成 `List`，运行时无法获取 `String` 类型参数
- 编译器在编译期做类型检查，保证类型安全
- 擦除后可通过反射绕过泛型约束（如向 `List<String>` 插入 `Integer`）

追问点：为什么不能 `new T()`？因为运行时不知道 `T` 的具体类型。可通过反射 `clazz.newInstance()` 或工厂模式解决。

问：泛型中 `<? extends T>` 和 `<? super T>` 的区别？PECS 原则？

答：– `<? extends T>`（上界）：只能 读取（Producer Extends），不能写入（除 `null`） – `<? super T>`（下界）：只能 写入 `T` 及其子类（Consumer Super），读取只能当 `Object`

PECS 原则：Producer Extends, Consumer Super。– 从集合 读 数据 → 用 `extends` – 往集合 写 数据 → 用 `super`

典型场景：`Collections.copy(List<? super T> dest, List<? extends T> src)`。

问：泛型和基本类型能直接用吗？如何处理？

答：不能。泛型参数必须是引用类型，基本类型需用包装类（`int` → `Integer`）。
自动装箱/拆箱带来性能开销；高并发场景可用 **特化集合**（如 `TIntArrayList`）或原始类型数组替代。

9. final 与 static

问：final 关键字可以修饰什么？各有什么含义？

答：

修饰对象	含义
类	不可被继承（如 <code>String</code> 、 <code>Integer</code> ）
方法	不可被重写（ <code>private</code> / <code>static</code> 隐式 <code>final</code> ）
变量	基本类型：值不可变；引用类型：引用不可变，对象内容可变
参数	方法内不可重新赋值

JMM 语义：`final` 字段在构造器完成后对其他线程可见（禁止重排序到构造器完成前），无需 `volatile`。

问：static 关键字的作用？静态代码块和构造器的执行顺序？

答：– **静态变量：**类级别，所有实例共享，类加载时初始化 – **静态方法：**属于类，不能访问实例成员，不能用 `this` – **静态代码块：**类加载时执行一次，用于初始化静态资源

执行顺序：父类静态块 → 子类静态块 → 父类构造器 → 子类构造器 → 实例代码块 → 构造方法体。

追问点：静态方法能否被重写？不能，只能被隐藏（`Hide`）。多态对静态方法无效，编译期绑定。

问：static 和 final 组合使用有什么场景？

答：– **常量：**`public static final int MAX_SIZE = 100` – **单例（饿汉式）：**`private static final Singleton INSTANCE = new Singleton()` – **工具类：**`public static final` 常量 + 私有构造器 + 全静态方法

10. 内部类

问：Java 内部类有哪些类型？各有什么特点？

答：

类型	说明	使用场景
成员内部类	非 static，持有外部类引用	逻辑紧密关联
静态内部类	static 修饰，不持有外部类引用	单例 DCL、Builder
局部内部类	方法/代码块内定义	临时辅助逻辑
匿名内部类	无类名，一次性使用	回调、事件监听

内存泄漏风险：非静态内部类隐式持有外部类 `this` 引用，若被长生命周期对象持有，外部类无法 GC。

问：为什么匿名内部类访问的局部变量必须是 final 或 effectively final？

答：匿名内部类可能在外部分方法返回后仍存活（如传入线程、回调）。若允许修改局部变量，会出现 **两个线程操作同一变量的并发问题**。

编译器将 effectively final 的局部变量 **复制一份** 到内部类，保证数据一致性。

11. BIO / NIO / AIO

问：BIO、NIO、AIO 的区别？

答：

模型	全称	特点	适用场景
BIO	Blocking IO	一连接一线程，阻塞等待	连接数少、简单
NIO	Non-blocking IO	多路复用 (Selector)，单线程处理多连接	高并发 (Netty、Tomcat NIO)
AIO	Asynchronous IO	操作系统完成 IO 后回调通知	连接数多、操作耗时

NIO 核心组件： – **Channel：**双向通道 (SocketChannel、FileChannel) – **Buffer：**数据容器 (ByteBuffer) – **Selector：**多路复用器，监听 Channel 事件 (OP_READ/OP_WRITE/OP_CONNECT/OP_ACCEPT)

问：NIO 的 Selector 工作原理？

答：1. 将 Channel 注册到 Selector，关注特定事件 2. 调用 `select()` 阻塞，直到有 Channel 就绪 3. 遍历 `selectedKeys()`，处理就绪 Channel 的读写

底层依赖 OS 的 **IO 多路复用**：Linux epoll、Mac kqueue、Windows select。

追问点：为什么 NIO 比 BIO 快？BIO 每个连接占一个线程，线程切换开销大；NIO 用少量线程管理大量连接，适合 C10K 问题。

12. 设计模式

问：单例模式有哪些实现方式？各有什么优缺点？

答：

方式	优点	缺点
饿汉式	简单、线程安全	类加载即创建，可能浪费
懒汉式（synchronized）	延迟加载	每次获取都加锁，性能差
DCL + volatile	延迟加载、性能好	需理解 JMM 禁止指令重排
静态内部类	延迟加载、线程安全	无法传参
枚举	防反射、防序列化破坏	不够灵活

推荐：一般场景用静态内部类；需防反射/序列化用枚举。

问：工厂模式和策略模式有什么区别？

答：– 工厂模式：关注 对象创建，将 new 的逻辑封装，调用方无需知道具体类 – 策略模式：关注 行为替换，定义算法族，运行时切换策略（如支付方式：微信/支付宝）

组合使用：工厂创建不同策略实例，策略接口定义行为，符合开闭原则。

问：代理模式在 Java 中有哪些实现？

答：– 静态代理：代理类手动编写，实现与目标相同的接口 – JDK 动态代理：基于接口，Proxy.newProxyInstance() + InvocationHandler – CGLIB 动态代理：基于继承，生成目标类子类（Spring AOP 无接口时用）

Spring AOP、MyBatis Mapper 接口、RPC 框架都大量使用代理模式。

13. 序列化

问：Java 序列化和反序列化的原理？serialVersionUID 的作用？

答：– 序列化：对象 → 字节流（实现 Serializable 接口） – 反序列化：字节流 → 对象

serialVersionUID：版本号，反序列化时校验类是否兼容。未显式声明则 JVM 根据类结构自动生成，类改动后 UID 变化导致 InvalidClassException。

建议：显式声明 private static final long serialVersionUID = 1L。

问：transient 关键字的作用？哪些字段不应序列化？

答：transient 修饰的字段不参与序列化，反序列化后为默认值（null / 0）。

不应序列化：密码、敏感信息、不可序列化的资源（Thread、Socket）、可重新计算的缓存字段。

问：如何防止序列化破坏单例？

答：1. 枚举单例（JVM 保证唯一） 2. 实现 `readResolve()` 方法，反序列化时返回已有实例

```
private Object readResolve() {
    return INSTANCE;
}
```

14. Java 21 新特性

问：Java 21 (LTS) 有哪些重要新特性？

答：1. 虚拟线程 (Virtual Threads)：轻量级线程，由 JVM 调度，适合 IO 密集型，可创建百万级线程 2. 结构化并发 (Structured Concurrency, 预览)：子任务生命周期绑定父任务，便于取消和错误传播 3. Record 模式匹配：switch 支持模式匹配，Record 解构 4. String Templates (预览)：字符串模板 5. Sequenced Collections：有序集合统一 API (`getFirst()`、`getLast()`) 6. 分代 ZGC：JDK 21 正式支持分代 ZGC，Young GC 更高效 7. Record 类：不可变数据载体，自动生成构造器、equals、hashCode、toString

问：Record 类是什么？和 Lombok @Data 的区别？

答：Record 是 JDK 16 正式引入的不可变数据类：

```
public record User(int id, String name) {}
// 自动生成：构造器、getter (id()、name())、equals、hashCode、toString
```

对比	Record	Lombok @Data
性质	语言级特性	编译期注解生成代码
可变性	不可变（字段 final）	可变
继承	隐式 final，不能继承	可继承
适用	DTO、值对象	通用 POJO

问：虚拟线程和平台线程（Platform Thread）的区别？

答：

对比	平台线程	虚拟线程
映射	1:1 映射 OS 线程	M:N 映射, JVM 调度
创建成本	约 1MB 栈空间	约几 KB
阻塞影响	阻塞 = 浪费 OS 线程	阻塞时挂起, 不占用 OS 线程
适用	CPU 密集型	IO 密集型 (Web 请求、DB 查询)

注意：虚拟线程中避免在 `synchronized` 块内做 IO（会 pin 到 OS 线程），推荐用 `ReentrantLock`。

15. Native 方法

问：Native 方法是什么？JNI 的作用？

答：`native` 修饰的方法由 本地代码 (C/C++) 实现，通过 **JNI (Java Native Interface)** 与 Java 交互。

典型应用： – `Object.hashCode()`、`Thread.start()` 等底层方法 – 调用 OS API、硬件驱动 – 性能关键路径（如压缩、加密算法）

流程：Java 声明 native 方法 → 生成 .h 头文件 → C/C++ 实现 → 编译为 .so/.dll → JVM 加载。

问：调用 Native 方法有什么风险？

答：1. **平台依赖：**不同 OS 需不同本地库 2. **内存管理：**C/C++ 手动管理内存，可能导致 JVM 崩溃（非 Java 异常） 3. **安全：**本地代码可绕过 Java 安全沙箱 4. **调试困难：**JVM 工具对 Native 栈支持有限

16. 其他高频

问：Java 中 AutoCloseable 和 try-with-resources 原理？

答：JDK 7 引入，自动关闭实现了 `AutoCloseable` 的资源（Connection、Stream 等）。

编译器将：

```
try (InputStream is = new FileInputStream("a.txt")) { ... }
```

转换为：try 块 + finally 中调用 `is.close()`，若 close 抛异常会 suppress 原异常。

问：Comparable 和 Comparator 在排序中的区别？

答：– `Comparable`：自然排序，实现 `compareTo()`，修改类本身 – `Comparator`：定制排序，独立比较器，不改源码

`TreeSet / TreeMap` 可用两者之一；`Collections.sort()` 优先用传入的 `Comparator`。

大厂追问

1. 值传递追问

「你说 Java 是值传递，那交换两个对象的内容为什么可以成功？」—— 因为传递的是引用副本，副本和原引用指向同一对象，通过副本可以修改对象内部状态，但无法让原引用指向另一个对象。

```
public static void swap(Person a, Person b) {  
    Person temp = a;  
    a = b; // 仅改变副本指向，不影响外部  
    b = temp;  
}  
// 外部引用不变；要真正交换需包装类或数组
```

2. String 常量池

`String s = new String("abc")` 创建了几个对象？—— 若常量池无 “abc”：堆中对象 + 常量池对象共 2 个；若已有则 1 个堆对象。`intern()` 会将堆中字符串放入常量池。

`s1 = "abc"; s2 = new String("abc"); s1 == s2 → false; s1 == s2.intern() → true。`

3. 装箱拆箱 NPE

```
Integer i = null;  
if (i == 1) { ... } // 拆箱时 NPE, 不是 false
```

比较包装类用 `Objects.equals(i, 1)` 更安全。

4. 泛型擦除实战

「List 和 List 在运行时是同一种类型吗？」—— 是，擦除后都是 `List`。因此不能 `list instanceof List <String>`，也不能创建泛型数组 `new T[10]`。

5. 反射性能

「反射为什么慢？」—— 涉及方法查找、安全检查、无法 JIT 内联优化。框架（Spring）启动时用反射，运行时多用缓存（如 CGLIB 生成的子类）弥补。

6. 虚拟线程陷阱

「虚拟线程适合所有高并发场景吗？」—— 不适合 CPU 密集型。大量计算会占用载体线程（Carrier Thread），且 `synchronized` 块会导致 pinning，失去虚拟线程优势。推荐 IO 阻塞处用 `ReentrantLock` 替代 `synchronized`。

7. 设计模式选型

「项目中用过哪些设计模式？为什么选它？」—— 准备 2~3 个真实案例：如策略模式做支付渠道切换、工厂+模板方法做消息推送、责任链做过滤器/拦截器。

8. 接口 default 方法冲突

类实现两个接口且 default 方法签名相同时，类必须重写该方法，否则编译错误。

9. 异常最佳实践

- 不要用异常控制业务流程
- catch 具体异常，避免 `catch (Exception e)` 吞掉所有
- 受检异常 vs 非受检：业务异常用 `RuntimeException`；可恢复的外部错误用 `Checked Exception`
- 资源关闭优先 `try-with-resources`